American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Subject: SOFTWARE QUALITY AND TESTING

Title: Automated Testing Tool

A SOFTWARE QUALITY AND TESTING

SUBMITTED TO

ABHIJIT BHOWMIK

Semester: Spring 25_26		**Section: B**
SN	Student Name	Student ID
1	MD. AL-IMRAN SAYEM	22-48023-2
2	MD. TANJIL TASHRIK ZIM	22-48021-2

1. Introduction:

Selenium is a popular automated testing tool used to test web applications. It can open a browser and do actions like a real user and perform actions like clicking, typing, and moving between pages.

Using Selenium, developers or testers can easily automate repetitive tasks. It supports different programming languages such as Python, Java, and C#.

Automated testing is a process where software testing is done using programs or scripts instead of doing it manually. Normally, a person checks everything step by step. But in automated testing, a script does those steps automatically. It saves time and reduces mistakes and we can run the same test again and again easily.

2. Tools Used

Selenium WebDriver – Used for automating web browser actions
Python – Programming language used to write test scripts
Google Chrome – Browser used for testing
Chrome Driver – Required to control Chrome browser using Selenium

3. Setup Process

In this step, Selenium was set up in the computer to perform automated testing.

- First, Python was installed and verified using command prompt.
- Then Selenium was installed using pip command (pip install selenium).
- After that, the Chrome browser version was checked.
- According to the Chrome version, ChromeDriver was downloaded and extracted.
- ChromeDriver was properly placed and configured.
- Finally, a test script was run to check whether Selenium is working correctly or not.

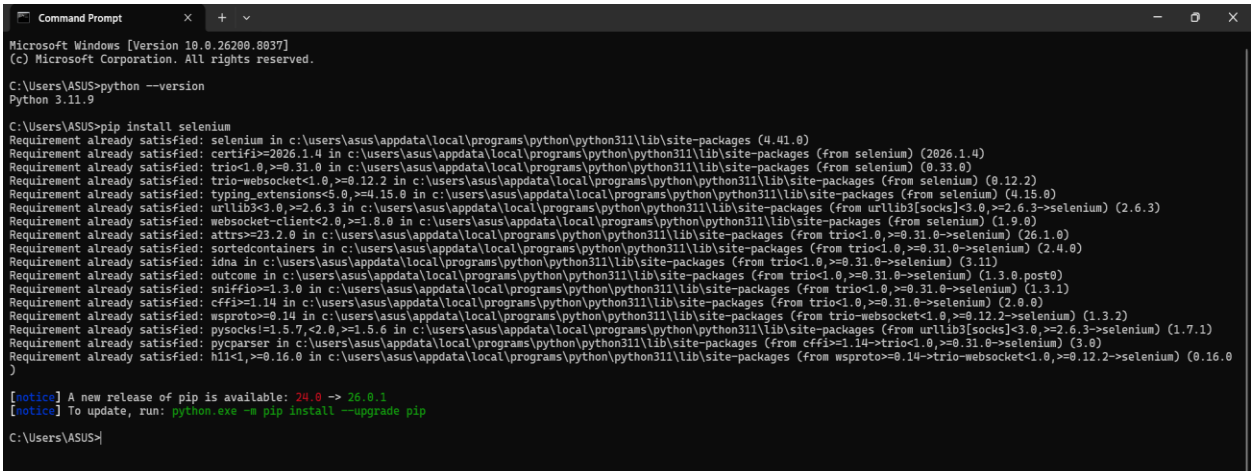


Fig 1. Check Python Version and Selenium install

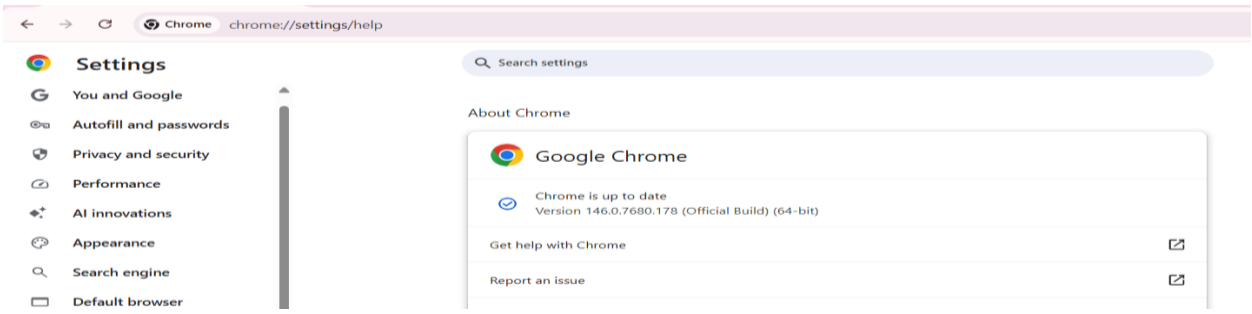


Fig 2. Chrome version check

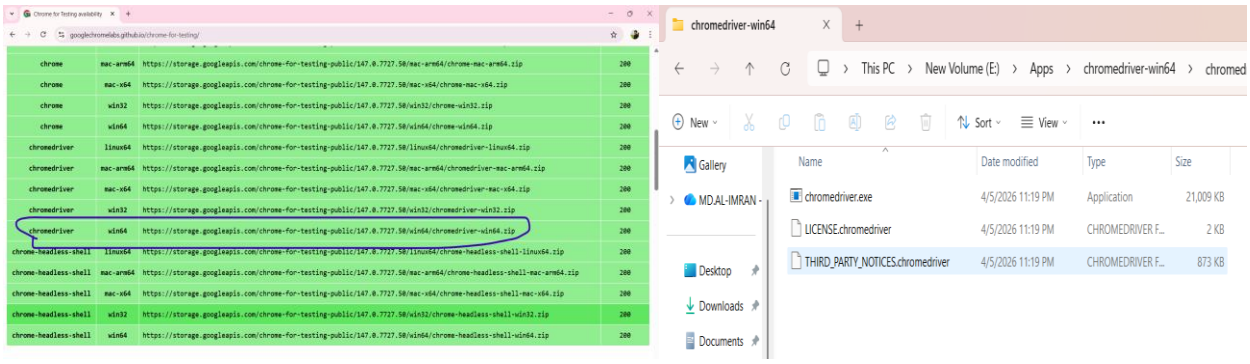


Fig 3. Chrome Driver download & Extract

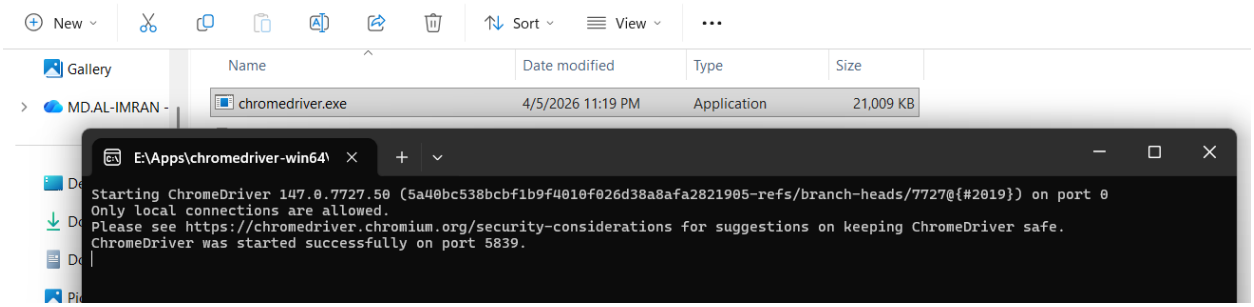


Fig 4. Chrome Driver Install

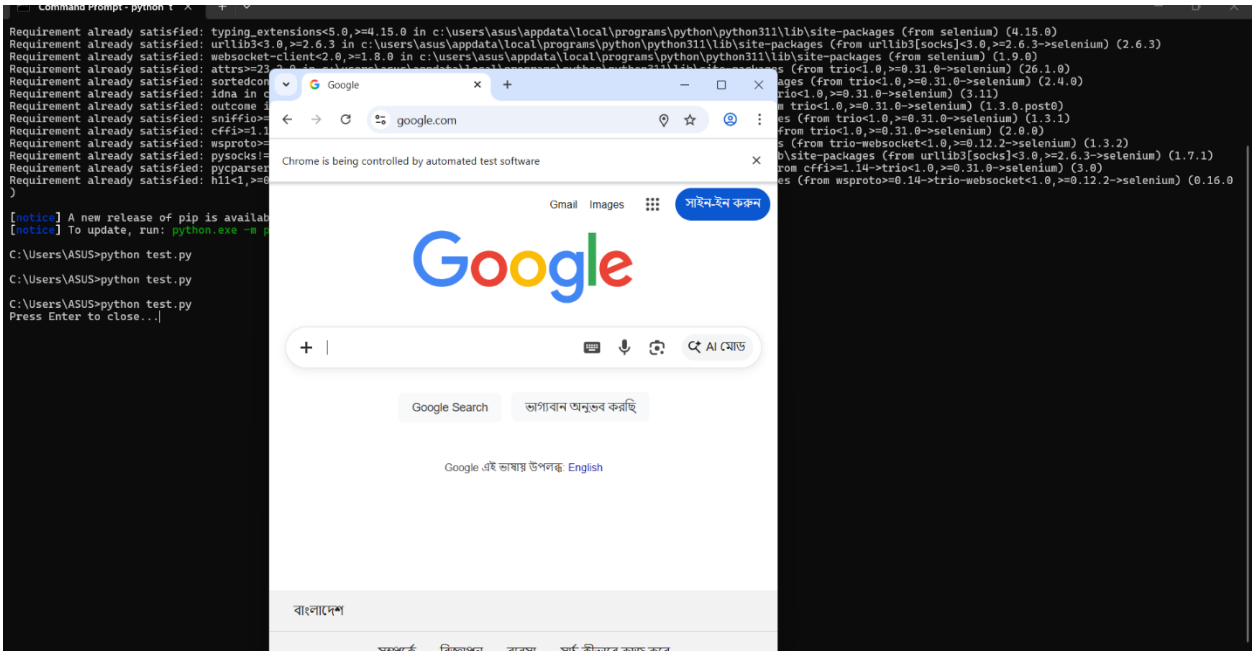


Fig 5. Check Selenium WORKING or Not

4. Testing Description

In this project, named **Bakery Management System**, we tested four main functionalities using Selenium.

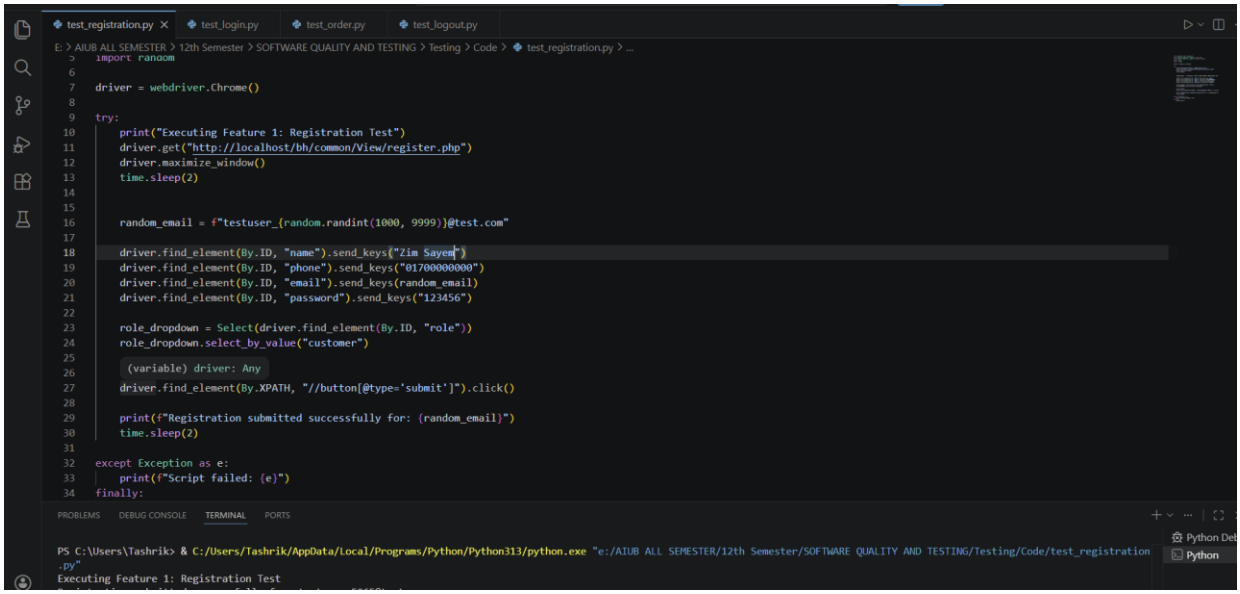
First, we tested the registration functionality. The user fills the registration form with name, phone, email, password, and role. The system checks if all fields are filled and validates the input properly. If everything is correct, the registration is completed successfully.

After that, we tested the login functionality. The user enters email and password to log in. If the credentials are correct, the system redirects the user to the dashboard based on their role.

Then, we tested the customer order feature. The customer selects a product and enters quantity to place an order. If the input is valid, the order is placed successfully.

Finally, we tested the logout functionality. The user clicks the logout link from their dashboard to securely end their session. The system terminates the active session and successfully redirects the user back to the login page.

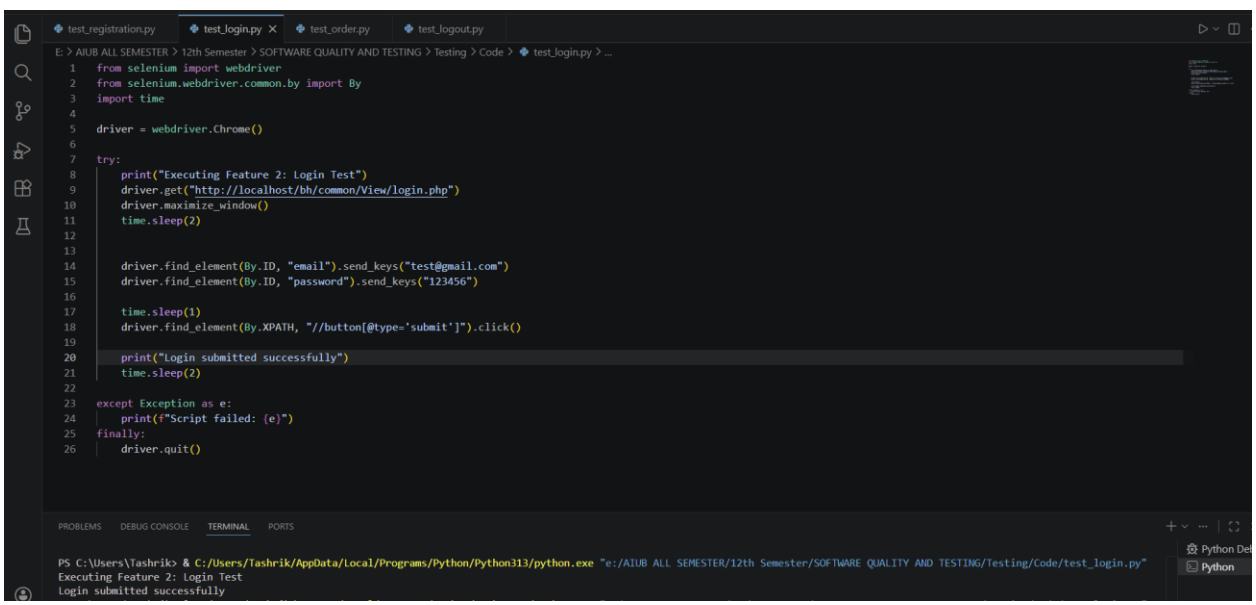
5.Result and process



```python
import random

driver = webdriver.Chrome()

try:
    print("Executing Feature 1: Registration Test")
    driver.get("http://localhost/bh/common/View/register.php")
    driver.maximize_window()
    time.sleep(2)


    random_email = f"testuser_{random.randint(1000, 9999)}@test.com"

    driver.find_element(By.ID, "name").send_keys("Zim Sayem")
    driver.find_element(By.ID, "phone").send_keys("01700000000")
    driver.find_element(By.ID, "email").send_keys(random_email)
    driver.find_element(By.ID, "password").send_keys("123456")

    role_dropdown = Select(driver.find_element(By.ID, "role"))
    role_dropdown.select_by_value("customer")

    (variable) driver: Any
    driver.find_element(By.XPATH, "//button[@type='submit']").click()

    print(f"Registration submitted successfully for: {random_email}")
    time.sleep(2)

except Exception as e:
    print(f"Script failed: {e}")
finally:
```

```
PS C:\Users\Tashrik> & C:/Users/Tashrik/AppData/Local/Programs/Python/Python313/python.exe "e:/AIUB ALL SEMESTER/12th Semester/SOFTWARE QUALITY AND TESTING/Testing/Code/test_registration.py"
Executing Feature 1: Registration Test
```

Fig 6: Feature 1 code (Automated Registration)



```python
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

try:
    print("Executing Feature 2: Login Test")
    driver.get("http://localhost/bh/common/View/login.php")
    driver.maximize_window()
    time.sleep(2)


    driver.find_element(By.ID, "email").send_keys("test@gmail.com")
    driver.find_element(By.ID, "password").send_keys("123456")

    time.sleep(1)
    driver.find_element(By.XPATH, "//button[@type='submit']").click()

    print("Login submitted successfully")
    time.sleep(2)

except Exception as e:
    print(f"Script failed: {e}")
finally:
    driver.quit()
```

```
PS C:\Users\Tashrik> & C:/Users/Tashrik/AppData/Local/Programs/Python/Python313/python.exe "e:/AIUB ALL SEMESTER/12th Semester/SOFTWARE QUALITY AND TESTING/Testing/Code/test_login.py"
Executing Feature 2: Login Test
Login submitted successfully
```

Fig 7: Feature 2 code (Login).

```python
try:

    print("Authenticating session...")
    driver.get("http://localhost/bh/common/View/login.php")
    driver.maximize_window()
    driver.find_element(By.ID, "email").send_keys("test@gmail.com")
    driver.find_element(By.ID, "password").send_keys("123456")
    driver.find_element(By.XPATH, "//button[@type='submit']").click()
    time.sleep(1) # Wait for redirect


    print("Executing Feature 3: Order Placement Test")
    driver.get("http://localhost/bh/customer/view/customer_dashboard.php?page=order")
    time.sleep(2)

    product_dropdown = Select(driver.find_element(By.NAME, "product_id"))
    product_dropdown.select_by_index(2)

    quantity_input = driver.find_element(By.NAME, "quantity")
    quantity_input.send_keys("1")

    time.sleep(1)
    driver.find_element(By.NAME, "place_order").click()

    print("Order placed successfully.")
    time.sleep(2)

except Exception as e:
    print(f"Script failed: {e}")
```

```
Executing Feature 2: Login Test
Login submitted successfully
PS C:\Users\Tashrik> & C:/Users/Tashrik/AppData/Local/Programs/Python/Python313/python.exe "e:/AIUB ALL SEMESTER/12th Semester/SOFTWARE QUALITY AND TESTING/Testing/Code/test_order.py"
Authenticating session...
Executing Feature 3: Order Placement Test
Order placed successfully.
PS C:\Users\Tashrik>
```

Fig 8: Feature 3 code (Order Placement)

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

try:

    print("Authenticating session...")
    driver.get("http://localhost/bh/common/View/login.php")
    driver.maximize_window()
    driver.find_element(By.ID, "email").send_keys("test@gmail.com")
    driver.find_element(By.ID, "password").send_keys("123456")
    driver.find_element(By.XPATH, "//button[@type='submit']").click()
    time.sleep(2)


    print("Executing Feature 4: Logout Test")
    logout_link = driver.find_element(By.LINK_TEXT, "Logout")
    logout_link.click()

    print("Logout executed successfully.")
    time.sleep(2)

except Exception as e:
    print(f"Script failed: {e}")
finally:
    driver.quit()
```

```
Executing Feature 3: Order Placement Test
Order placed successfully.
PS C:\Users\Tashrik> & C:/Users/Tashrik/AppData/Local/Programs/Python/Python313/python.exe "e:/AIUB ALL SEMESTER/12th Semester/SOFTWARE QUALITY AND TESTING/Testing/Code/test_logout.py"
Authenticating session...
Executing Feature 4: Logout Test
Logout executed successfully.
PS C:\Users\Tashrik>
```

Fig 9: Feature 4 code (Session Logout)

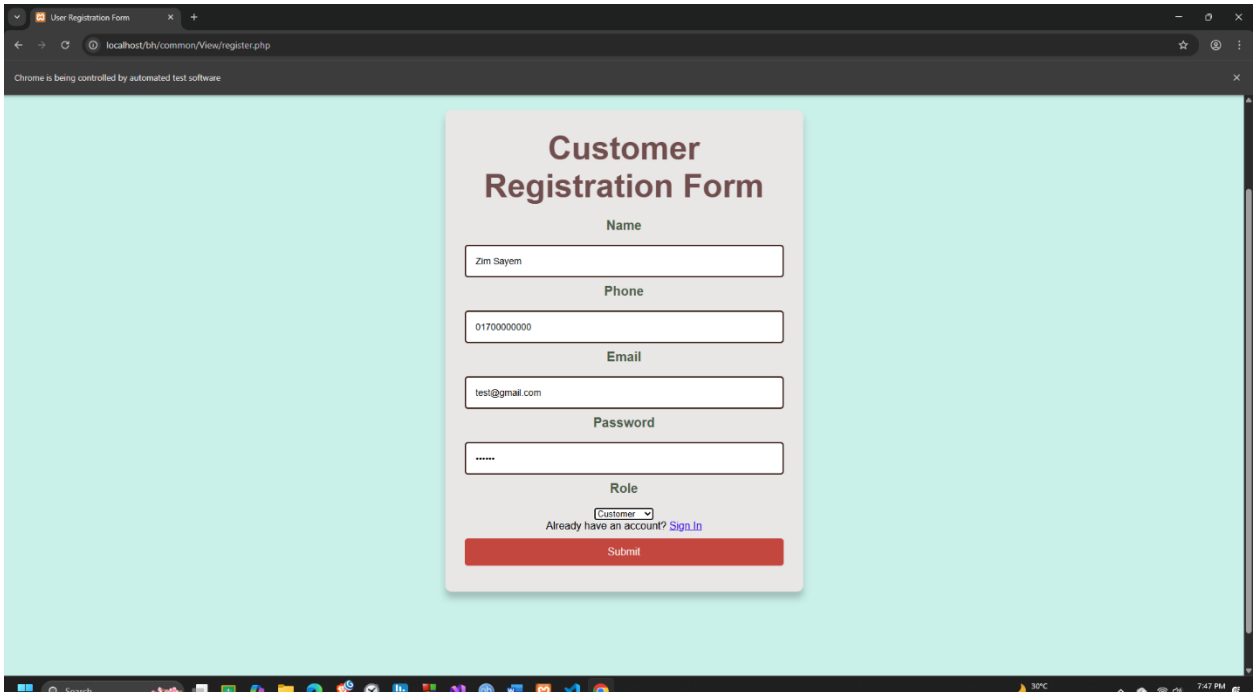


Fig 10: Testing Feature 1 (Registration). The script automatically types our test data into the form.

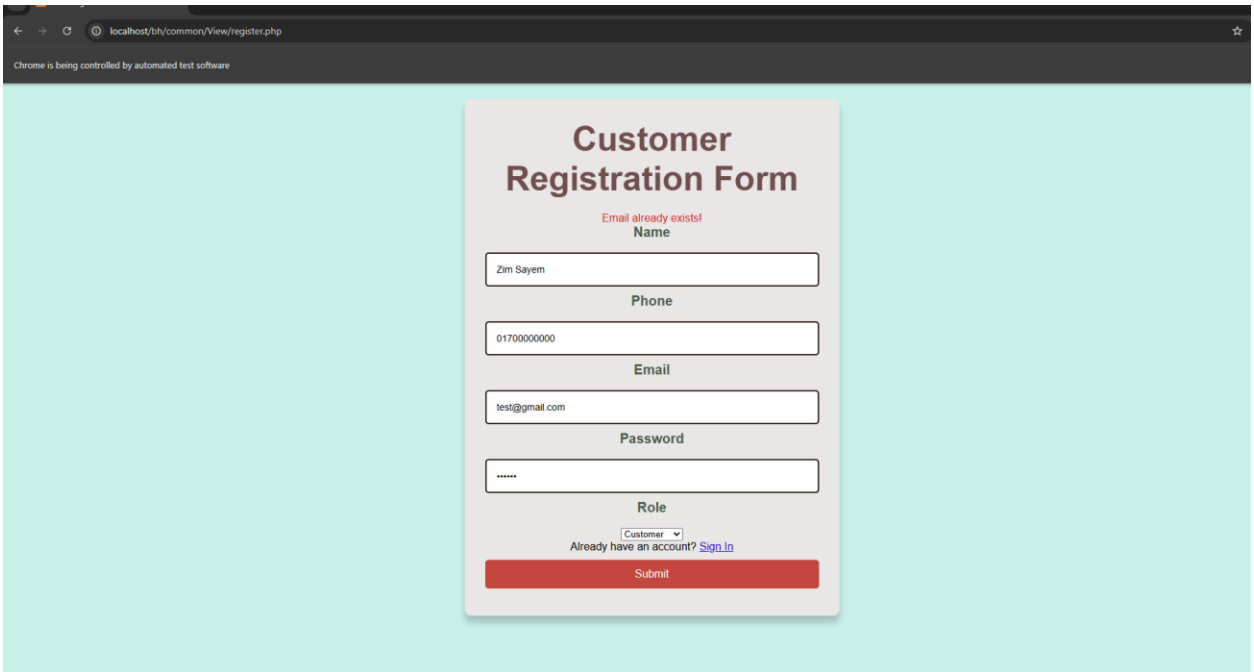


Fig 11: Negative Testing. Making sure the system correctly blocks duplicate emails and shows an error message when the script tries to register twice.

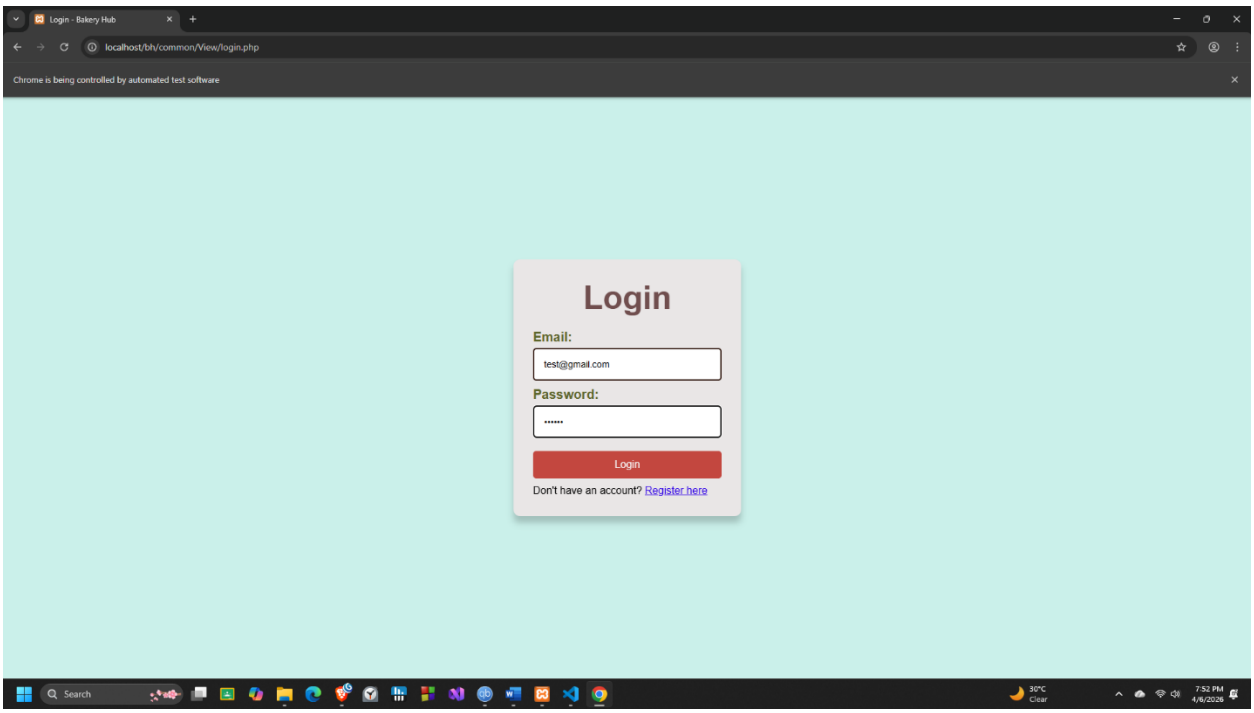


Fig 12: Testing Feature 2 (Login). The script logging into the bakery system using the new test account.

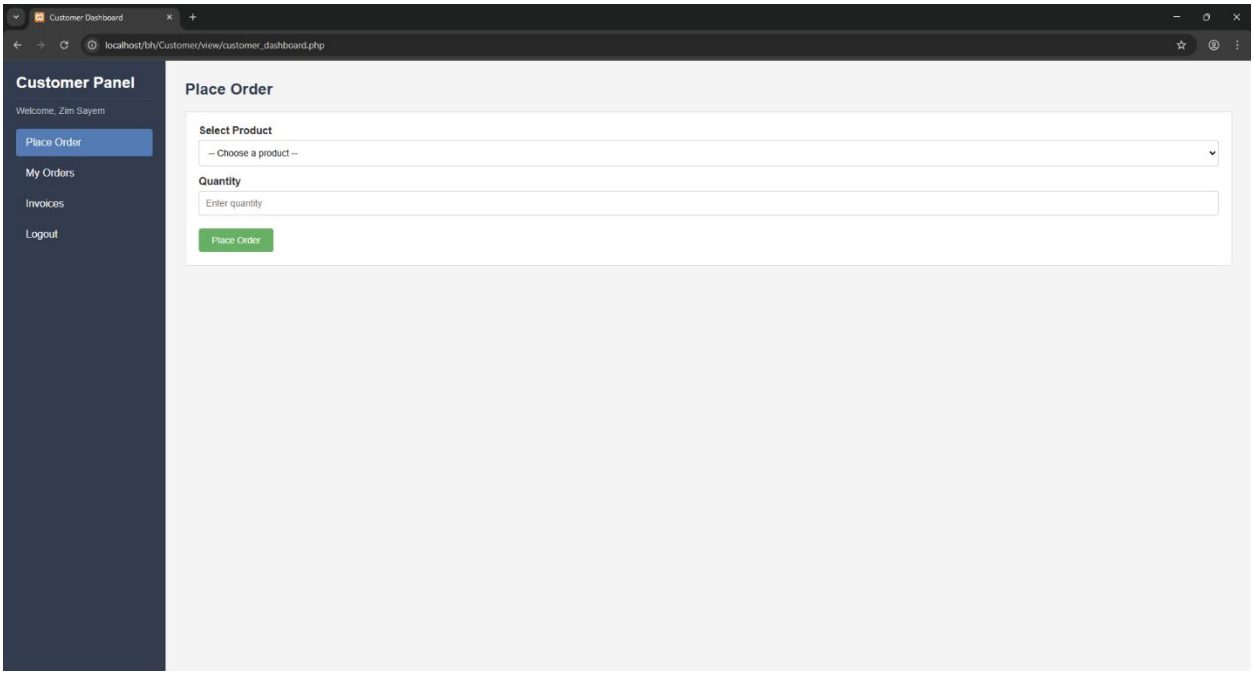


Fig 13: Testing Feature 3 (Order). The automated browser goes to the customer dashboard and gets ready to place an order.

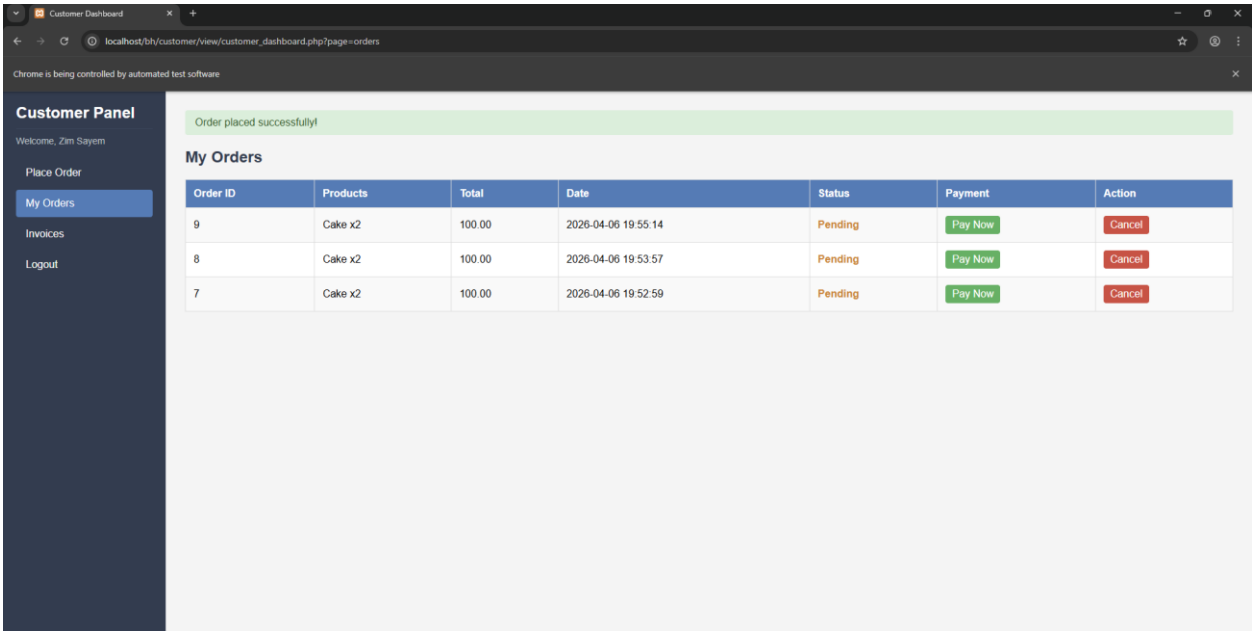


Figure 14: Order placed successfully. New items added to the order list.

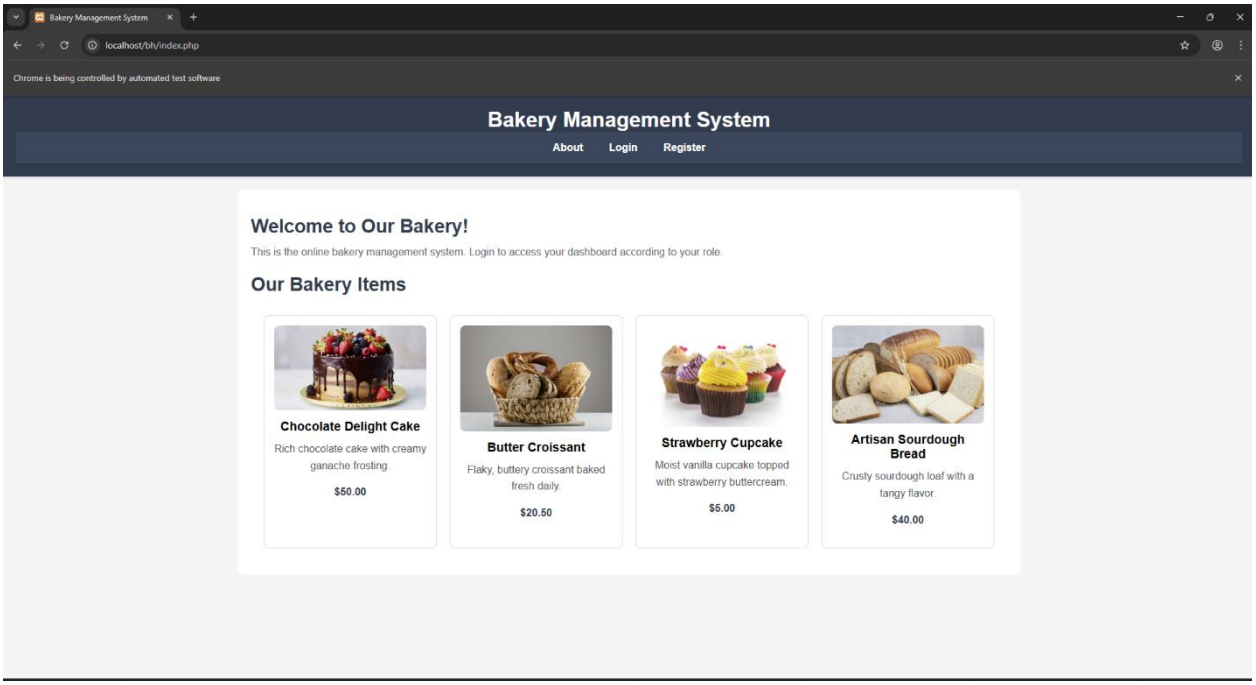


Fig 15: Testing Feature 4 (Logout). The script clicks the logout button and successfully goes back to the home page.

Conclusion: This project gave us practical experience with automated web testing using Selenium. By successfully writing and running scripts for the registration, login, ordering, and logout features, we learned how automation can speed up the testing process, reduce manual effort, and help ensure a web application works correctly.